Sri Lanka Institute of Information Technology



Lab Submission
<Lab sheet 08>

<IT26100802>

<Wickramasinghe V.I.>

Discrete Mathematics | IT1160

B.Sc. (Hons) in Information Technology

In [1]:

```
import networkx as nx
import matplotlib.pyplot as plt

T = nx.DiGraph()

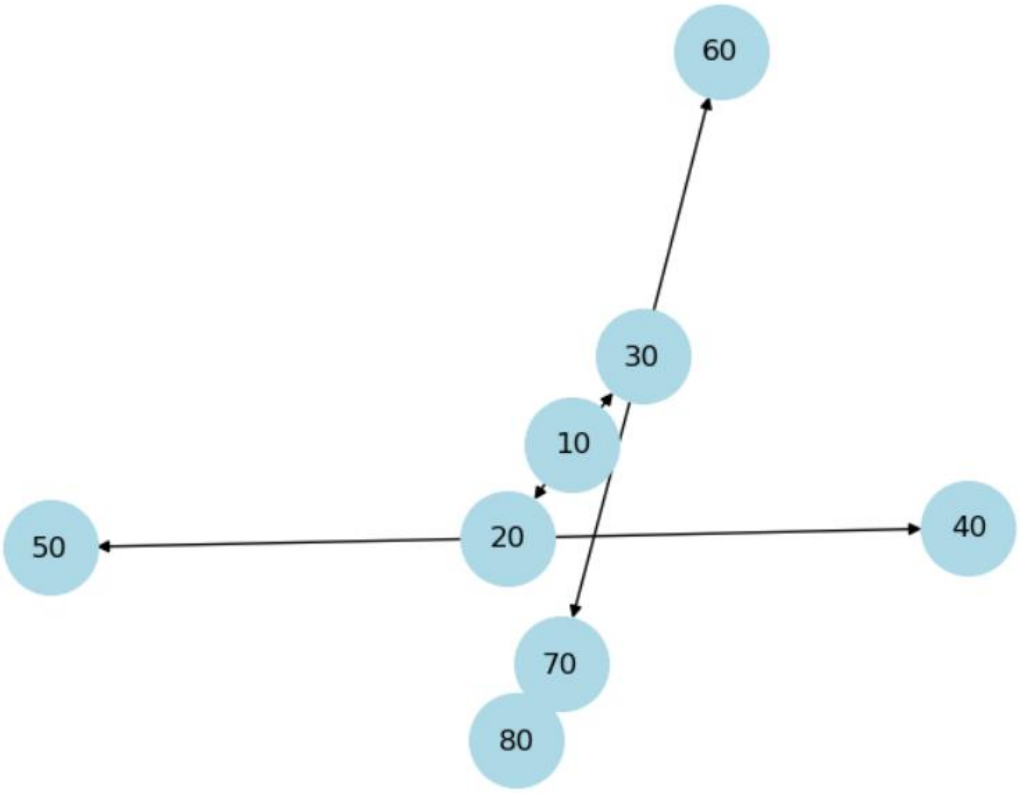
edges = [(10,20), (10,30), (20,40),
         (20,50), (30,60), (30,70), (70,80)]

T.add_edges_from(edges)

pos = nx.spring_layout(T)

nx.draw(T, pos, with_labels=True,
        node_color="lightblue",
        node_size=1500, arrows=True)

plt.show()
```


60
30
10
50
20
40
70
80

In [2]:
```
parent = list(T.predecessors(40))
print("Parent of 40:", parent[0])
```

```
Parent of 40: 20
```

In [3]:
```
root = 10

height = max(
    nx.shortest_path_length(T, source=root).values()
)

print("Height of tree:", height)
```

```
Height of tree: 3
```

In [4]:
```
leaves = [node for node in T.nodes()
          if T.out_degree(node) == 0]

print("Leaf nodes:", leaves)
```

```
Leaf nodes: [40, 50, 60, 80]
```

In [5]:
```
root = [node for node in T.nodes()
        if T.in_degree(node) == 0]

print("Root node:", root[0])
```

```
Root node: 10
```

In [6]:
```
parent = list(T.predecessors(50))[0]

siblings = list(T.successors(parent))
siblings.remove(50)

print("Siblings of 50:", siblings)
```

```
Siblings of 50: [40]
```

In [7]:
```
depth = nx.shortest_path_length(T, 10, 70)

print("Depth of 70:", depth)
```

```
Depth of 70: 2
```

```python
import matplotlib.pyplot as plt

T = nx.DiGraph()

edges = [
    ('ER', 'TRIAGE'),
    ('ER', 'ICU'),
    ('ICU', 'SURG'),
    ('ICU', 'CARD'),
    ('SURG', 'NEURO')
]

T.add_edges_from(edges)

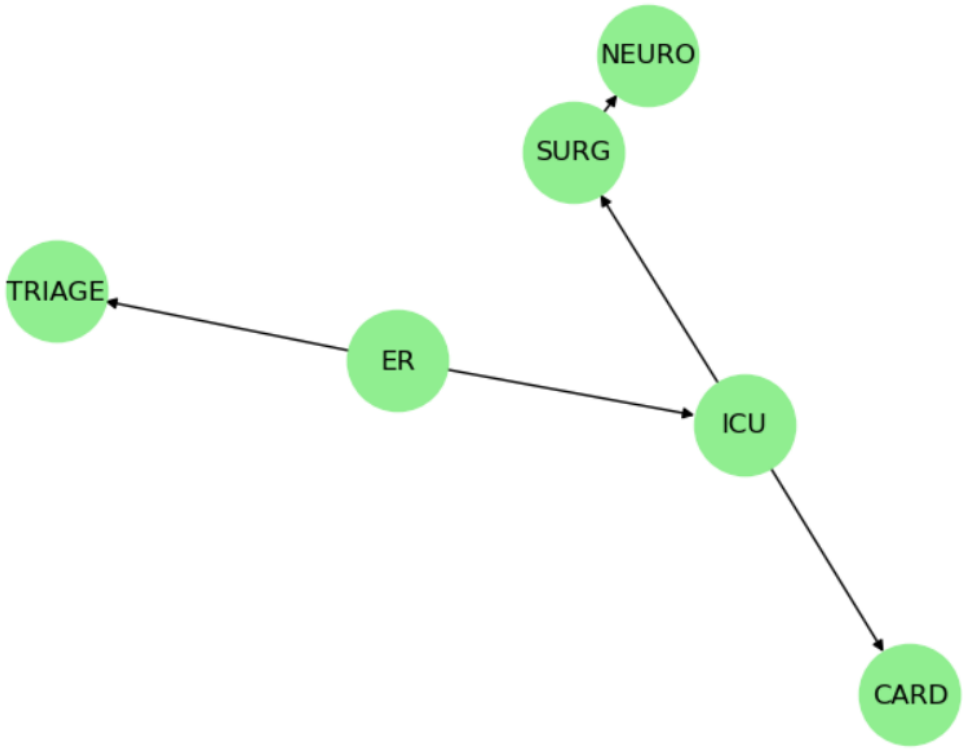
pos = nx.spring_layout(T)

nx.draw(T, pos, with_labels=True,
        node_color="lightgreen",
        node_size=2000, arrows=True)

plt.show()

is_binary = all(T.out_degree(node) <= 2
                for node in T.nodes())

print("Is Binary Tree:", is_binary)
```


NEURO
SURG
TRIAGE
ER
ICU
CARD

In [9]:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

edges = [
    ('A', 'B', 2),
    ('A', 'D', 6),
    ('B', 'C', 3),
    ('B', 'D', 8),
    ('C', 'D', 5)
]

G.add_weighted_edges_from(edges)

pos = nx.spring_layout(G)

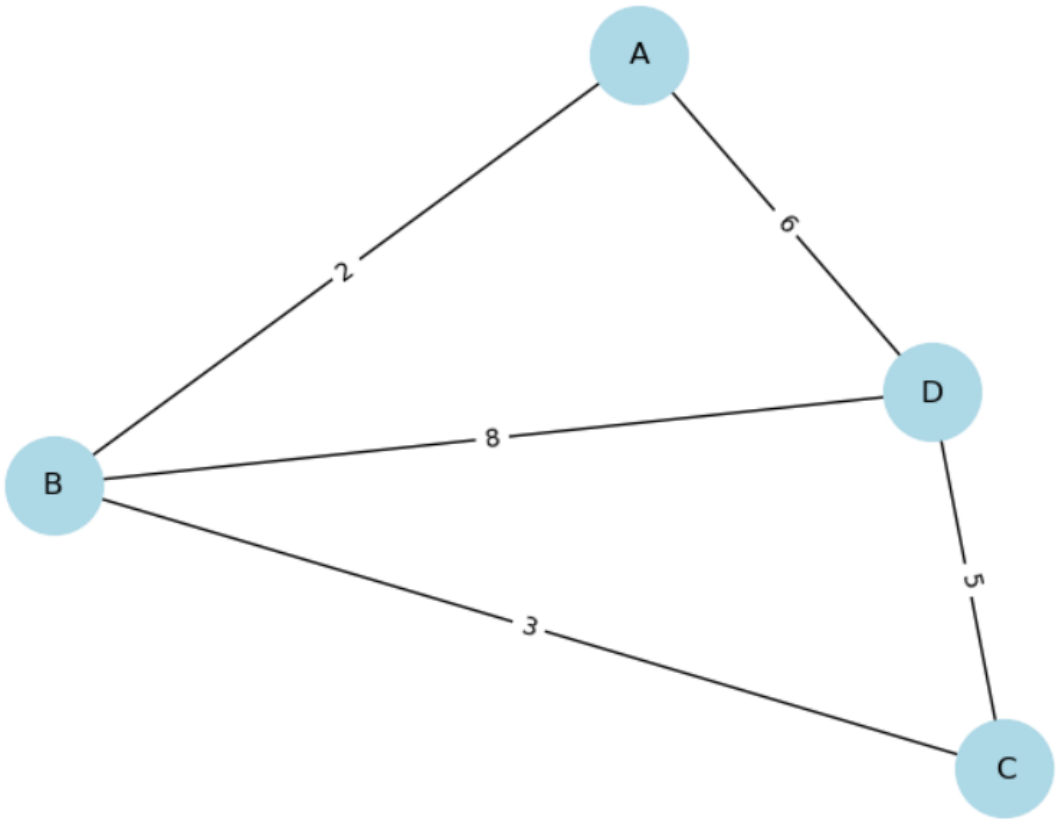
nx.draw(G, pos, with_labels=True,
        node_color="lightblue",
        node_size=1500)

labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

plt.show()

MST = nx.minimum_spanning_tree(G, algorithm="prim")

print("MST Edges:", list(MST.edges(data=True)))
print("Total Cost:", MST.size(weight='weight'))
```


A
2
6
D
B
8
5
3
C

```
MST Edges: [('A', 'B', {'weight': 2}), ('B', 'C', {'weight': 3}), ('D', 'C', {'weight': 5})
Total Cost: 10.0
```

n [10]:

```python
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

edges = [
    ('a', 'b'),
    ('a', 'e'),
    ('e', 'd'),
    ('b', 'd'),
    ('e', 'b'),
    ('b', 'c'),
    ('d', 'c')
]

G.add_edges_from(edges)

pos = nx.spring_layout(G)

nx.draw(G, pos, with_labels=True,
        node_color="lightblue",
        node_size=1500)

plt.show()

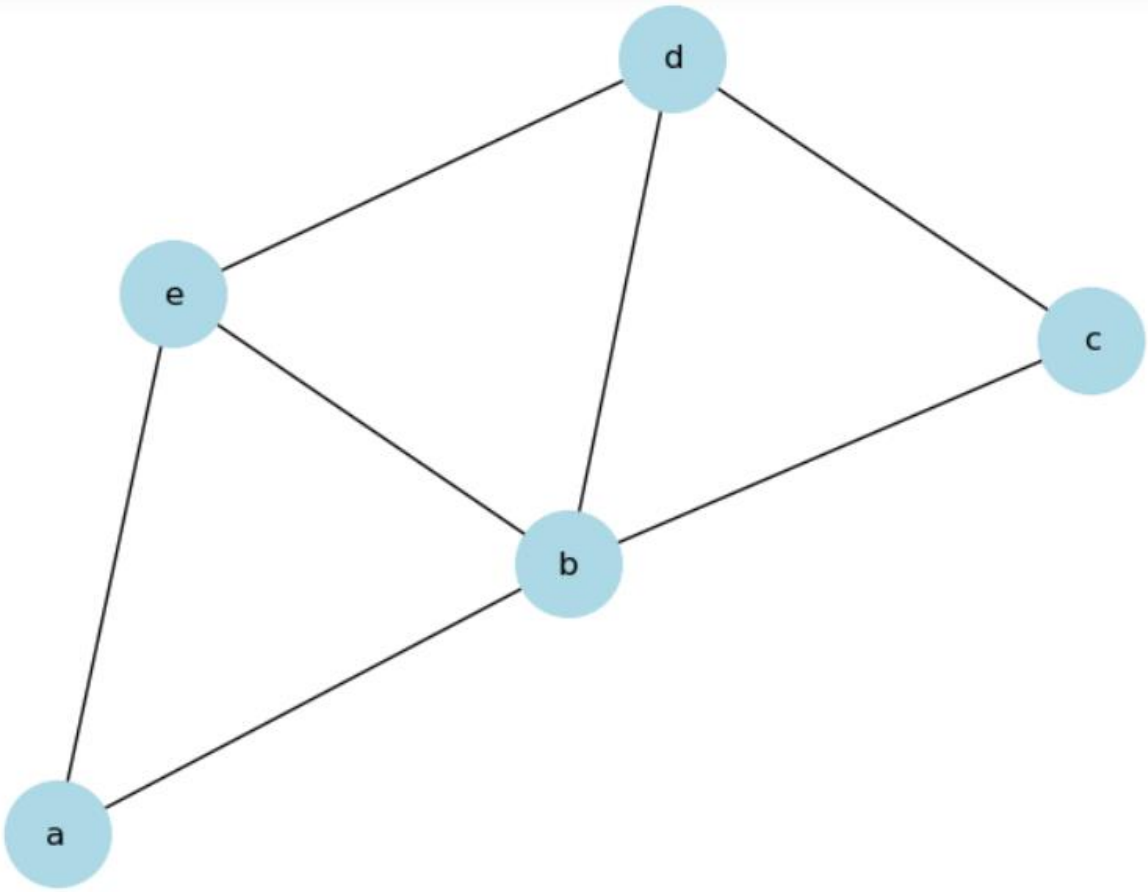
bfs = list(nx.bfs_tree(G, source='a'))
```

```python
nx.draw(G, pos, with_labels=True,
        node_color="lightblue",
        node_size=1500)

plt.show()

bfs = list(nx.bfs_tree(G, source='a'))

print("BFS Traversal:", bfs)
```




```
BFS Traversal: ['a', 'b', 'e', 'd', 'c']
```

In [11]:

```
import networkx as nx
import matplotlib.pyplot as plt

T = nx.DiGraph()

edges = [
    (1, 2), (1, 8), (1, 9),
    (2, 6), (2, 5),
    (5, 3), (5, 4),
    (8, 7)
]

T.add_edges_from(edges)

pos = nx.spring_layout(T)

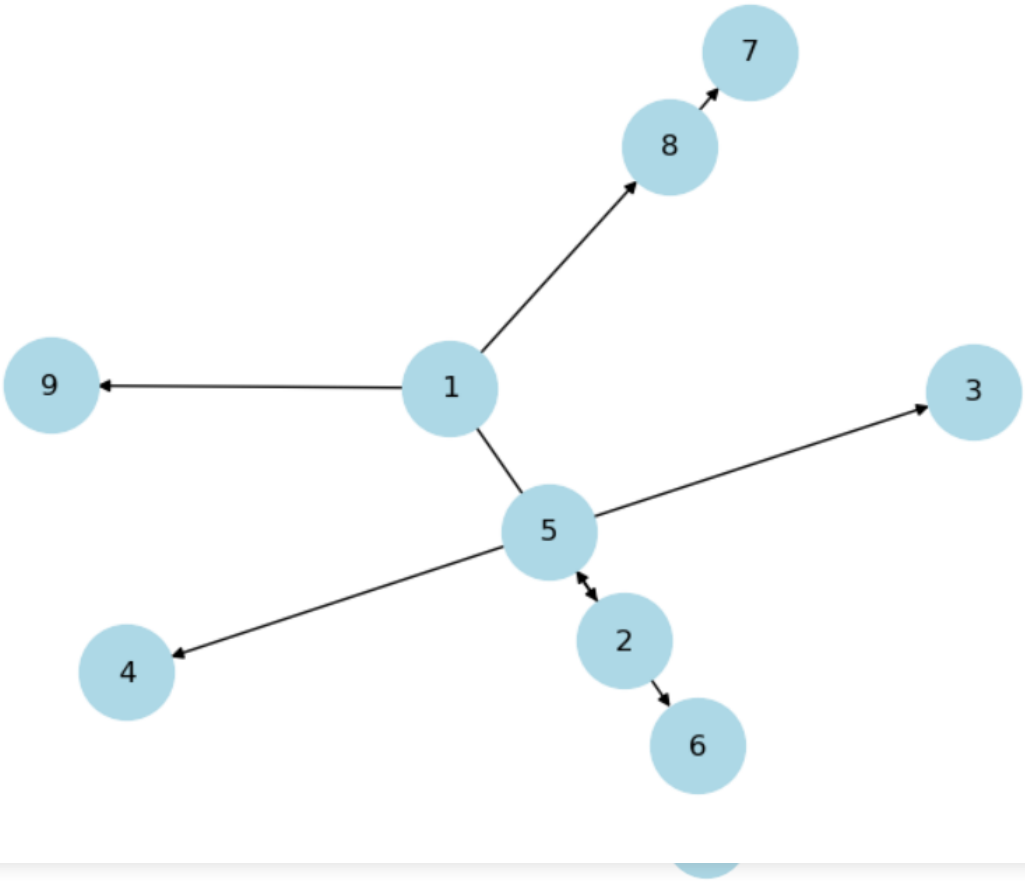
nx.draw(T, pos, with_labels=True,
        node_color="lightblue",
        node_size=1500, arrows=True)

plt.show()

parent = list(T.predecessors(7))
print("Parent of 7:", parent[0])

height = max(
    nx.shortest_path_length(T, source=1).values()
)

print("Height of tree:", height)
```




```
Parent of 7: 8
Height of tree: 3
```

In [12]:
```python
def is_full_binary_tree(tree):

    for i in range(len(tree)):

        if tree[i] is None:
            continue

        left = 2 * i + 1
        right = 2 * i + 2

        left_exists = (left < len(tree) and tree[left] is not None)
        right_exists = (right < len(tree) and tree[right] is not None)

        if left_exists != right_exists:
            return False

    return True


tree = [1, 2, 3, 4, 5, None, None]

print("Is Full Binary Tree:", is_full_binary_tree(tree))
```

```
Is Full Binary Tree: True
```

In [13]:

```
import networkx as nx
import matplotlib.pyplot as plt

T = nx.DiGraph()

edges = [
    ('ADMIN', 'SCIENCE'),
    ('ADMIN', 'ARTS'),
    ('SCIENCE', 'PHYSICS'),
    ('SCIENCE', 'CHEMISTRY'),
    ('ARTS', 'HISTORY'),
    ('ARTS', 'LANGUAGES')
]

T.add_edges_from(edges)

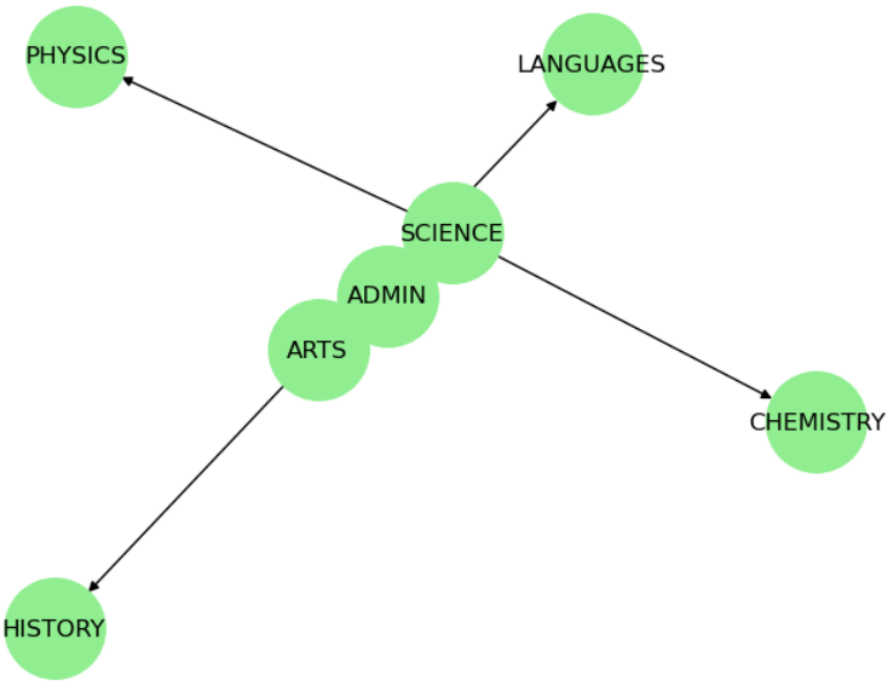
pos = nx.spring_layout(T)

nx.draw(T, pos, with_labels=True,
        node_color="lightgreen",
        node_size=2500, arrows=True)

plt.show()

is_full = all(
    T.out_degree(node) in [0, 2]
    for node in T.nodes()
)

print("Is Full Binary Tree:", is_full)
```




```
Is Full Binary Tree: True
```

In [14]:
```python
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

edges = [
    ('a', 'b', 5),
    ('b', 'c', 4),
    ('a', 'd', 2),
    ('d', 'g', 6),
    ('g', 'h', 4),
    ('h', 'i', 2),
    ('c', 'f', 3),
    ('f', 'i', 4),
    ('d', 'e', 7),
    ('e', 'f', 1),
    ('b', 'e', 5),
    ('e', 'h', 3),
    ('d', 'b', 3),
    ('d', 'h', 8),
    ('b', 'f', 6),
    ('h', 'f', 4)
]

G.add_weighted_edges_from(edges)

pos = nx.spring_layout(G)
```

```python
pos = nx.spring_layout(G)

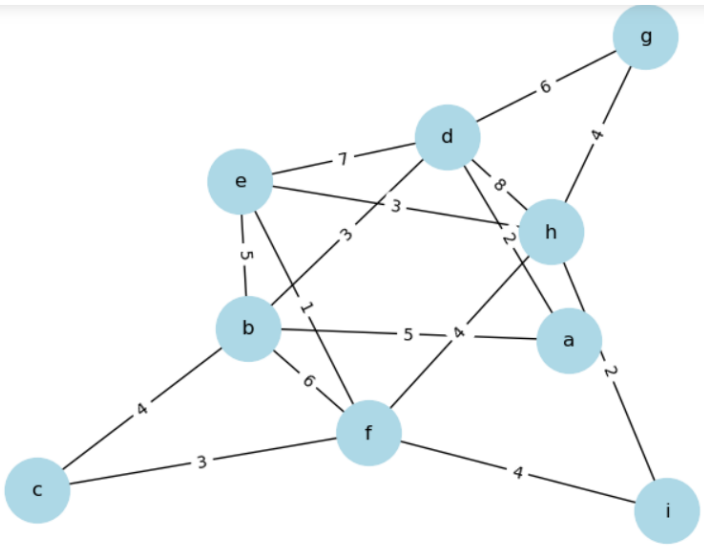
nx.draw(G, pos, with_labels=True,
        node_color="lightblue",
        node_size=1500)

labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

plt.show()

MST = nx.minimum_spanning_tree(G, algorithm="prim")

print("MST Edges:", list(MST.edges(data=True)))
print("Total Cost:", MST.size(weight='weight'))
```




```
MST Edges: [('a', 'd', {'weight': 2}), ('b', 'c', {'weight': 4}), ('b', 'd', {'weight': 3}), ('c', 'f', {'weight': 3}), ('g',
'h', {'weight': 4}), ('h', 'e', {'weight': 3}), ('h', 'i', {'weight': 2}), ('f', 'e', {'weight': 1})]
Total Cost: 22.0
```

In [15]:

```python
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

edges = [
    ('a', 'b'),
    ('a', 'c'),
    ('a', 'd'),
    ('b', 'c'),
    ('b', 'd'),
    ('b', 'g'),
    ('c', 'd'),
    ('c', 'e'),
    ('d', 'f')
]

G.add_edges_from(edges)

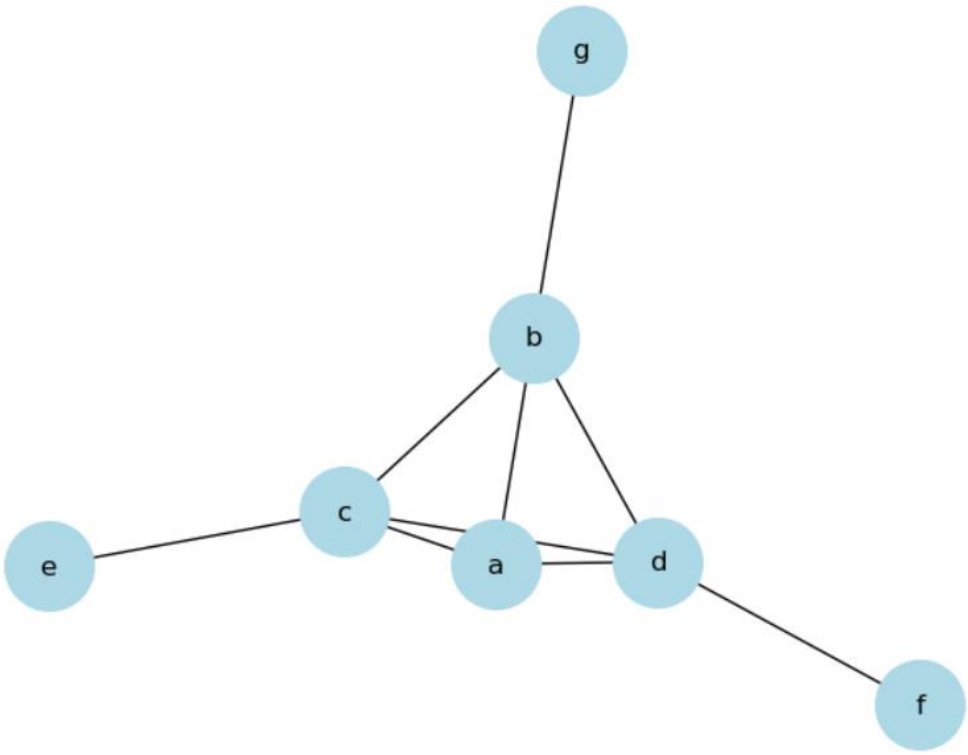
pos = nx.spring_layout(G)

nx.draw(G, pos, with_labels=True,
        node_color="lightblue",
        node_size=1500)

plt.show()

dfs = list(nx.dfs_tree(G, source='a'))

print("DFS Traversal:", dfs)
```




```
DFS Traversal: ['a', 'b', 'c', 'd', 'f', 'e', 'g']
```